Production-Modus

Lernunterlage: Umgebungen in ASP.NET Core – warum der Dauerdienst nicht im Development laufen soll

1 Worum es geht

ASP.NET Core kennt **Umgebungen** (Environments). Welche gilt, steht in der Umgebungsvariable `ASPNETCORE_ENVIRONMENT`. Üblich sind **Development**, **Staging** und **Production**. Ohne Angabe ist es Production, also die sichere Voreinstellung.

Bis zum 02.11.2026 lief der Dauerdienst als Development, nur damit die User Secrets gelesen werden. Das Security-Review (REVIEW.md, Finding **M-1**) hat das bemängelt: Sicherheitsentscheidungen hingen an einem Schalter, der für die Entwicklung gedacht ist.

2 Was sich je nach Umgebung ändert

Verhalten	Development (app-starten.sh)	Production (Dauerdienst)
User Secrets	liest ASP.NET Core automatisch	nur mit Schalter HomeControl:UseUserSecrets
Refresh-Cookie	ohne Secure (läuft auch über http)	Secure: Browser schickt es nur sicher
API-Doku /scalar, /openapi	an, ohne Anmeldung	aus (404), keine Angriffskarte
Ohne Jwt:SigningKey	Zufallsschlüssel + Warnung im Log	App startet nicht (Exception)
Demo-Daten (SeedDemoData)	erlaubt	nie
appsettings.Development.json	wird geladen	wird nicht geladen

> Faustregel: Development ist bequem, Production ist sicher. Was dauerhaft läuft und von anderen Geräten erreichbar ist, läuft als Production.

3 Das Problem: Wo kommen die Geheimnisse her?

JWT-Schlüssel, Tuya-Zugang, Blink und Agenten-Schlüssel liegen in den **User Secrets** (`~/.microsoft/usersecrets/<id>/secrets.json`, Rechte 600). Die liest ASP.NET Core von sich aus **nur im Development**. Zwei Wege standen zur Wahl:

Weg	Vorteil	Nachteil
A: User Secrets ausdrücklich einbinden (gewählt)	Geheimnisse an einer Stelle, alle *-einrichten.sh funktionieren weiter	Nur sinnvoll auf dem eigenen Rechner
B: Umgebungsvariablen-Datei für systemd	Üblicher Weg auf Servern und in Docker	Geheimnisse doppelt, müssten bei jedem Update abgeglichen werden

Für den OVH-Server (Variante B in BETRIEB.md) kommt später Weg B: Dort gibt es keine User Secrets.

4 Der Code

Program.cs: ausdrücklich einbinden, nur mit Schalter

```
var builder = WebApplication.CreateBuilder(args);

if (!builder.Environment.IsDevelopment()
    && builder.Configuration.GetValue<bool>("HomeControl:UseUserSecrets"))
{
    builder.Configuration.AddUserSecrets<Program>(optional: false);
    builder.Configuration.AddEnvironmentVariables();   // wieder nach hinten
}
```

- **Ganz am Anfang:** Gleich danach liest Program.cs den Connection String und den JWT-Schlüssel. Die Quelle muss vorher da sein.
- **optional: false:** Fehlt die Datei, bricht der Start ab. Lieber ein klarer Fehler als ein Dienst ohne Zugangsdaten.
- **Reihenfolge der Quellen:** Die zuletzt hinzugefügte Quelle gewinnt. AddUserSecrets hängt hinten an, damit würden die Secrets sogar Umgebungsvariablen überstimmen. Deshalb die Umgebungsvariablen noch einmal anhängen: Der Connection String aus dem Dienst gilt weiter.

```
appsettings.json → appsettings.Production.json → Umgebungsvariablen → Kommandozeile
                                  → User Secrets (neu) → Umgebungsvariablen (nochmal)
                                                              ▲ gewinnt
```

Der Schalter im Dienst (dienst-einrichten.sh)

```
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=HomeControl__UseUserSecrets=true
```

Zwei Unterstriche `__` in einer Umgebungsvariable entsprechen dem Doppelpunkt in der Konfiguration: `HomeControl__UseUserSecrets` = `HomeControl:UseUserSecrets`.

Vorher prüfen statt hinterher suchen

```
SECRETS="$(dotnet user-secrets list --project backend/HomeControl.Api 2>/dev/null || true)"
if ! grep -q '^Jwt:SigningKey = ' <<<"$SECRETS"; then
    echo "Fehler: In den User Secrets fehlt Jwt:SigningKey." >&2
    exit 1
fi
```

- **Vor dem Anhalten des Dienstes:** Fehlt der Schlüssel, läuft der alte Dienst einfach weiter.
- **In eine Variable statt per Pipe:** Mit `set -o pipefail` könnte `grep -q` die Pipe früh schließen. Der erste Befehl bekäme dann ein Signal, und die Prüfung schlüge fälschlich fehl. Die Werte erscheinen so auch nicht auf dem Bildschirm.

5 Tests: ProductionModeTests

```
public class ProductionApiFactory : HomeControlApiFactory
{
    protected override Dictionary<string, string?> Settings => new()
        { ["Jwt:SigningKey"] = SigningKey };

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        base.ConfigureWebHost(builder);
        builder.UseEnvironment("Production");   // später = gewinnt
    }
}
```

Test	prüft
Api_Dokumentation_ist_aus	/scalar und /openapi/v1.json antworten mit 404
Refresh_Cookie_ist_Secure	Set-Cookie für hc_refresh enthält Secure und HttpOnly
Ohne_JWT_Schluessel_startet_die_App_nicht	Start wirft eine Exception, die Jwt:SigningKey nennt

HandleCookies = false beim Test-Client: Sonst verarbeitet der Client das Cookie selbst, und man sieht die Eigenschaften im Header nicht mehr so gut.

6 Und http://127.0.0.1:5200?

Ein Secure-Cookie schickt der Browser nur über sichere Verbindungen. Der Zugriff über Tailscale ist HTTPS, das passt. Für `localhost` und `127.0.0.1` machen Browser eine Ausnahme: Diese Adressen gelten als „potenziell vertrauenswürdig“, denn die Daten verlassen den Rechner nie. Deshalb funktioniert die Anmeldung dort weiter.

7 Kurzantworten für Rückfragen

Frage	Antwort
Was ist der Unterschied zwischen Development und Production?	Development ist bequem (API-Doku, Secrets, Ersatzschlüssel), Production ist sicher (Secure-Cookie, keine Doku, harter Abbruch).
Warum User Secrets im Production?	Damit die Geheimnisse an einer Stelle bleiben. Nur auf dem eigenen Rechner und nur mit ausdrücklichem Schalter.
Was passiert ohne JWT-Schlüssel?	Die App startet nicht. Ein Zufallsschlüssel würde nach jedem Neustart alle Tokens ungültig machen und Fehler verdecken.
Wo ist jetzt die API-Doku?	Nur noch beim Entwickeln: app-starten.sh oder dotnet run, dann /scalar.
Wie macht man das auf einem Server?	Geheimnisse als Umgebungsvariablen oder Docker-Secrets, Umgebung Production, Schalter aus.

8 Merksätze

- Umgebung über ASPNETCORE_ENVIRONMENT, Standard ist Production.
- Was dauerhaft läuft, läuft als Production.
- Konfiguration: Die zuletzt hinzugefügte Quelle gewinnt.
- __ in Umgebungsvariablen = : in der Konfiguration.
- Fehlende Geheimnisse früh und laut melden, nicht mit Ersatzwerten verdecken.